

Six_Modern_TTR

March 26, 2025

1 Comparative Analysis of Literary Translations in the Odyssey: Type-Token Ratio in Depth

This is a comparative Type-Token Ratio (TTR) analysis of six translations spanning roughly the last century: 1919–2018.

1.1 Road Map

1.1.1 I. Libraries, files, and paths

1.1.2 II. The Texts

1. Bibliographic information about the translators
 - a) Name, date, publisher, digital source
 - b) Etymologic bias, character names and place names
2. The translators at a glance tokenwise
 - a) Length and distribution
 - Total token-words ratio
 - Book distribution
 - Choosing a baseline: Murray(?)
 - b) Token quality check
 - DataFrame of firstlines (books, authors)
 - Visual auditing Book 6
 - Purity index
 - by book
 - aggregated

1.1.3 III. TTR Analysis

1. All-in, straightforward model
 - a) TTR Computation

- b) Shapiro-Wilk test to check for normality
- c) One-way ANOVA for overall differences
- d) Pairwise t-test using Bonferroni correction
- e) Measuring effect size using Cohen's d
- 2. Adaptive models
 - a) Mixed-Effects model: author fixed effect / book as random effect
 - b) Standardized TTR:
 - c) Moving-average TTR: translation as temporal change
- 3. Supplement models
 - a) Lexical Density
 - b) Diachronic analysis
 - c) Semantic fields:

1.1.4 IV. Discussing Results

1.2 I. Libraries, files, and paths

```
[1]: ##### Other libraries
import sys
import os

import ast
from collections import Counter

import re
import nltk
```

```
[2]: ##### Pandas set-up
import numpy as np
import pandas as pd
```

```
[3]: ##### Visualization
sys.path.append('/Users/debr/English-Homer')
import matplotlib.pyplot as plt
import seaborn as sns
import bard_visualization as viz # My Visualization library
```

Bard viz is on the stage!

```
[4]: # Import my functions
sys.path.append('/Users/debr/English-Homer/functions')
```

```
import e_nlp as e
```

e enters! »----> use e.<function>

e.g., for pipeline:

```
nlp = e.NLPPipeline(language="english")
```

Stopwords customized:

Added: {'one', 'nine', 'ten', 'n', 'four', 'eight', 'two', '', "'and",
'seven', 'three', 'six', 'five'}

Removed: {''}

Total stopwords: 211

Punctuation customized:

Keeping: {'', '-'}

Additional removals: {'-', "'", '!', '...', '!', '\\', '-', '!'}

Punctuation to be removed: !"#%&'()*+,-./:;<=>?@[\\]^_`{|}~-''"...

[nltk_data] Downloading package punkt_tab to /Users/debr/nltk_data...

[nltk_data] Package punkt_tab is already up-to-date!

```
[5]: ##### Cell display options
pd.set_option("display.max_rows", 2000) # Increase row display
pd.set_option("display.max_colwidth", 200) # Prevent truncation of long values
pd.set_option("display.precision", 2) # Display numbers with two decimal places
pd.set_option("display.float_format", "{:.2f}".format) # Format float numbers
pd.set_option("display.max_columns", None) # Show all columns
#pd.set_option("display.expand_frame_repr", False) # Prevent wrapping in
↳DataFrames
pd.set_option("display.width", 1000) # Prevent wrapping in DataFrames
pd.set_option("colheader_justify", "left")
```

```
[6]: ##### File management
# TO UPDATE
nb_id = "Six_Modern_TTR"
odysseys = ["AT_Murray", "Fitzgerald", "Lattimore", "Fagles", "Wilson", "Green"]

# Paths
output_path = f"/Users/debr/English-Homer/{nb_id}/"
os.makedirs(os.path.dirname(output_path), exist_ok=True)

output_path_plots = f"{output_path}/{nb_id}_plots/"
os.makedirs(os.path.dirname(output_path_plots), exist_ok=True)
```

```
[7]: # Daframe with all the odysseys
dfs = []

for odyssey in odysseys:
```

```

    filepath = f"/Users/debr/odysseys_en/dataframed/Odyssey_{odyssey}_DataFrame.
↪CSV"
    temp_df = pd.read_csv(filepath)
    dfs.append(temp_df) # Append it to the list

df = pd.concat(dfs, axis=0, ignore_index=True)

df["text"] = df["text"].apply(ast.literal_eval)
df = df[['author', 'book_num', 'text', 'tokens', 'num_words', 'num_tokens']]
df["diff"] = df["num_words"] - df["num_tokens"]
e.check_df(df)

```

No missing values

```
df columns: Index(['author', 'book_num', 'text', 'tokens', 'num_words',
'num_tokens', 'diff'], dtype='object')
```

Shape: (144, 7)

```
[8]: df.sample(4, random_state=4)
```

```

[8]:      author      book_num text
tokens
num_words  num_tokens  diff
128      Green      9      [Then resourceful Odysseus responded to him,
saying:\n, "Alkinoos, lord, most distinguished among all peoples,\n, it is
indeed a good thing to listen to such a minstrel\n, as this man is, who rese...
['resourceful', 'odysseus', 'responded', 'saying', 'alkinoos', 'lord',
'distinguished', 'among', 'peoples', 'indeed', 'good', 'thing', 'listen',
'minstrel', 'man', 'resembles', 'gods', 'singing', ... 5950      2786
3164
5      AT_Murray      6      [So here Odysseus slept, overcome by sleep and\n,
toil; but Athena went off to the demos and city of\n, the Phaeacians - a people
who used to live in the fair town of Hypereia, near\n, the lawless...
['odysseus', 'slept', 'overcome', 'sleep', 'toil', 'athena', 'went', 'demos',
'city', 'phaeacians', 'people', 'used', 'live', 'fair', 'town', 'hypereia',
'near', 'lawless', 'cyclopes', 'cyclopes',... 3470      1492      1978
61      Lattimore 14      [But Odysseus himself left the harbor and ascended a
rugged\n, path, through wooded country along the heights, where Athene\n, had
indicated the noble swineherd, who beyond others\n, cared for the...
['odysseus', 'left', 'harbor', 'ascended', 'rugged', 'path', 'wooded',
'country', 'along', 'heights', 'athene', 'indicated', 'noble', 'swineherd',
'beyond', 'others', 'cared', 'house', 'properties... 6328      2728
3600
29      Fitzgerald      6      [Far gone in weariness, in oblivion, \n, the noble
and enduring man slept on; \n, but Athena in the night went down the land \n, of
the Phaiakians, entering their city. \n, In days gone by, these ... ['far',

```

'gone', 'weariness', 'oblivion', 'noble', 'enduring', 'man', 'slept', 'athena',
 'night', 'went', 'land', 'phaiakians', 'entering', 'city', 'days', 'gone',
 'men', 'held', 'hypereia', 'count... 2907 1454 1453

1.3 II. The Texts

1.3.1 1. Bibliographic information about the translators

Here is a brief description of each translation, starting with bibliographic data: a) Name, date, publisher, source, etc.; followed by labelling their b) Etymologic bias, regarding characters and places names.

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Note	Bibliography
Augustus Taber Murray	1919	Modern	1910-1940	FALSE	Harvard UP, Loeb	1919	Scaife Viewer, Tufts Univer- sity	Link	“No more faith-ful trans-la-tion of A. Homer was (Au-ever gus-made us and Taber), its trans-el-e-tor. gance Lon-match led: its William fi- Hein-delity mann; Homer New for- York: mu- G.P. laic Put-ep- nam’s i- Sons, thets, 1919. phrases, and sen-tences were con-sis-tently ren-dered, and his ar-ti-fi-cial amal-gam of di-alects and ar-choic	

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Note	Bibliography
Robert Fitzger- ald	1961	(Ur)	Post-1960	TRUE	Doubleday & Com- pany, Inc.	Second 1963 Edi- tion	Internet Archive	Link		“Written in <i>The</i> very <i>Odyssey</i> . read-Fitzger- ald, <i>ald</i> , AmerRobert, i- <i>trans</i> - can <i>la</i> - “plainor. style” <i>New</i> lan- <i>York</i> : guageDou- (Car- <i>ble</i> - olyn <i>day</i> Clark& Best <i>Com</i> - Amerpany, i- <i>Inc.</i> , can <i>1961</i> Po- etry, 2020) “ProseHomer. like <i>The</i> verse <i>Odyssey</i> . transLat- la- <i>ti</i> - tion” <i>more</i> , (Car-Rich- olyn <i>mond</i> , Clarktrans- Best <i>la</i> - Amentor. i- <i>New</i> can <i>York</i> : Po- <i>Dou</i> - etry, <i>ble</i> - 2020) <i>day</i> & <i>Com</i> - <i>pany</i> , <i>Inc.</i> , <i>1965</i>
Richmond Latti- more	1965	(Ur)	Post-1960	TRUE	Farrar, Straus & Giroux	First 1965 Edi- tion	Harper Collins E-Books	NONE		“ProseHomer. like <i>The</i> verse <i>Odyssey</i> . transLat- la- <i>ti</i> - tion” <i>more</i> , (Car-Rich- olyn <i>mond</i> , Clarktrans- Best <i>la</i> - Amentor. i- <i>New</i> can <i>York</i> : Po- <i>Dou</i> - etry, <i>ble</i> - 2020) <i>day</i> & <i>Com</i> - <i>pany</i> , <i>Inc.</i> , <i>1965</i>

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Notes	Bibliography
Robert Fagles	1996	Postmodern	1800-1999	FRU	Penguin	First 1996 Edition	PDF SCAN	NONE		<i>Homer. The Odyssey. Fa- gles, Robert, trans- la- tor. Uni- ver- sity of Cali- for- nia Press, 1996</i>
Emily Wilson	2017	Contemporary	2000-	FRU	W.W. Norton & Com- pany Inc.	First 2017 Edition	PDF SCAN	NONE		<i>Homer. The Odyssey. Wil- son, Emily, trans- la- tor. W.W. Nor- ton & Com- pany Inc., 2017</i>

Author	Year	Period	Span	Verse	Publisher	Edition	Source	URL	Notes	Bibliography
Peter Green	2018	Contemporary	2000	RU	University of California Press	Digital	EBSCOhost	Link		<i>Homer. The Odyssey. Green, Peter, translator. University of California Press, 2018</i>

Use of Latin vs Greek form for characters' name by translator. There is an inconsistency in their use. Up until Fagles, every translator used a mix of direct Greek transliteration and the conventional name passed on by the earlier tradition.

- **Murray:** Lean to use Greek (Telemakhos, Menelaos) yet more popular names (Odysseus, Athena) remain in their Latin form
- **Fitzgerald:** Opts for transliterating Greek phonetics, like Murray Telemakhos but to an extreme in Akhilleus, however others retain their popular form.
- **Lattimore:** Less Greek than Fitzgerald (Telemachus) but still Athene and Achilleus
- **Fagles:** Experiment less with character names and stick to the convention
- **Wilson:** Follows the same names as Fagles
- **Green:** Returns to a Greek flavor by including Greek diacritics to the names: Achillēs, Menelaōs, Tēlemachos, Athēnē

The names of places follow the same pattern. This will be analyzed thoroughly in the Name Entity Recognition step of the project.

2. The translators at a glance tokenwise a) Length and distribution

- Total token-words ratio
- Book distribution
- Choosing a baseline: Murray(?)

```
[9]: # Create a color mapping for authors
authors = df["author"].unique()
palette = {author: color for author,
           color in zip(authors, viz.color_palette.values())}
```

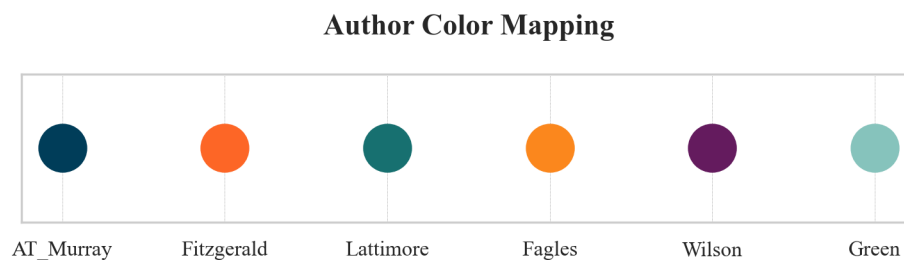
```

# Plot the color mapping
plt.figure(figsize=(6, 1))

for i, (author, color) in enumerate(palette.items()):
    plt.scatter(i, 0, color=color, s=300)

plt.xticks(range(len(palette)), palette.keys(), fontsize=8)
plt.yticks([])
plt.title("Author Color Mapping", fontsize=11)
plt.savefig(f"{output_path_plots}author_color_mapping-{nb_id}.png")
plt.show()

```



```

[10]: # Compute diff
df["diff"] = df["num_words"] - df["num_tokens"]

# Aggregate sum per author
agg_diff = df.groupby("author")["diff"].sum()
mean_agg_diff = agg_diff.mean()

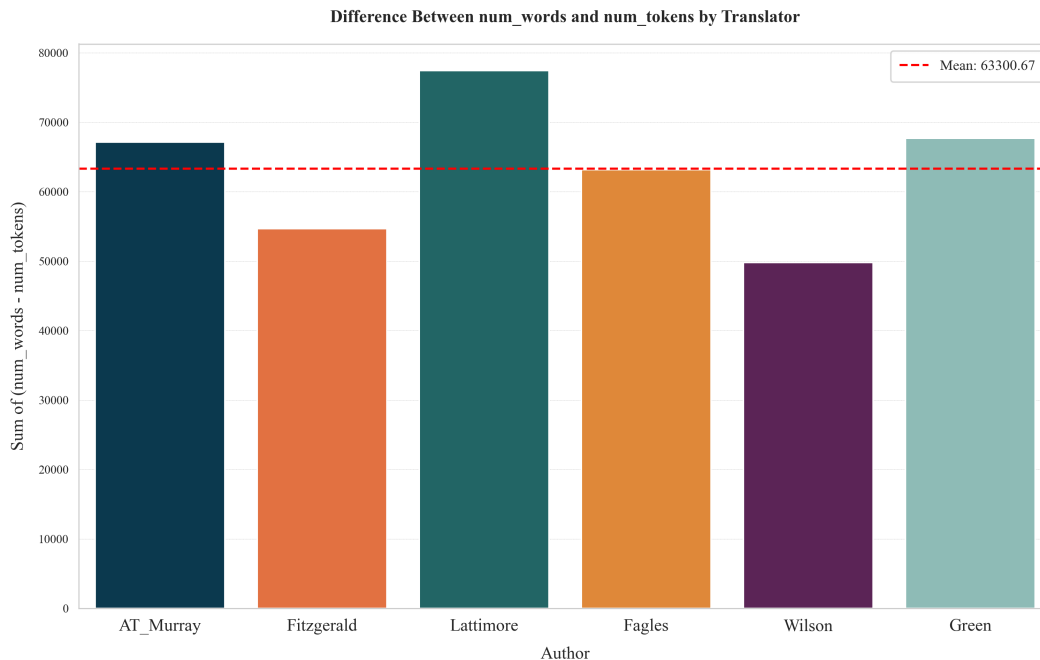
# Plot
sns.barplot(data=df, x="author", y="diff", estimator=sum, errorbar=None,
            hue="author", palette=palette) # Explicit color mapping

# Mean horizontal line
plt.axhline(mean_agg_diff, color="red", linestyle="--",
            label=f"Mean: {mean_agg_diff:.2f}")

# Labels and title
plt.xlabel("Author")
plt.ylabel("Sum of (num_words - num_tokens)")
plt.title("Difference Between num_words and num_tokens by Translator")

```

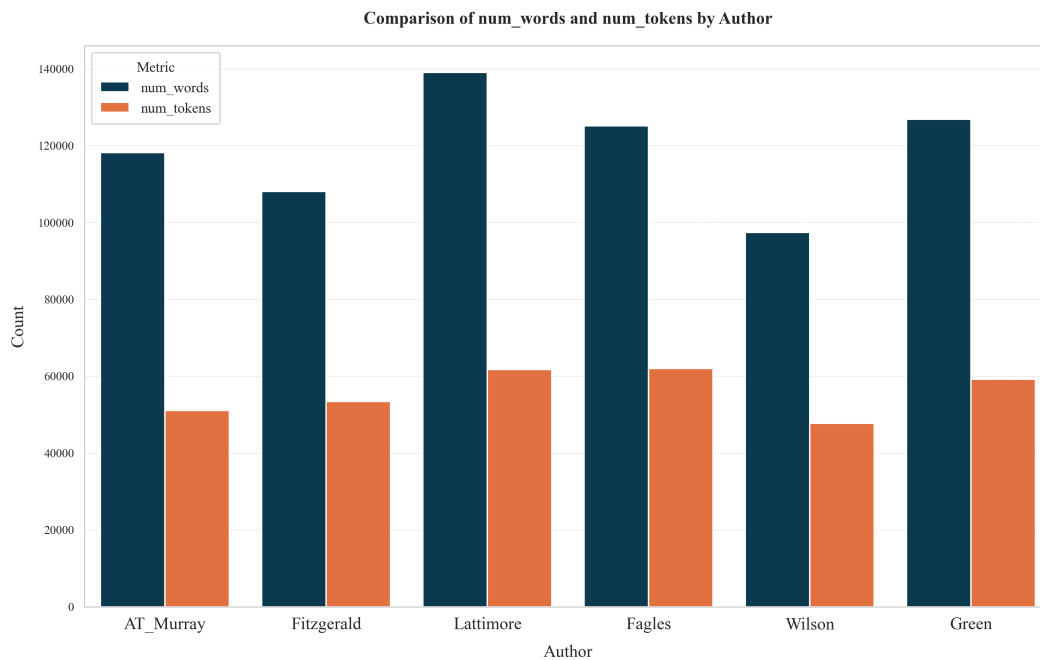
```
plt.legend()
plt.xticks() # Rotate x-axis labels for better readability
plt.savefig(output_path_plots + f"diff_words_tokens-{nb_id}.png") # Save plot
# Show plot
plt.show()
```



```
[11]: # Side by side comparison of num_words and num_tokens by translator
# Reshape the DataFrame to a long format
df_melted = df.melt(id_vars=["author"], value_vars=["num_words", "num_tokens"],
                    var_name="Metric", value_name="Count")

# Create the bar plot
sns.barplot(data=df_melted, x="author", y="Count", hue="Metric",
            estimator=sum, errorbar=None)

# Customize labels
plt.xticks(rotation=0)
plt.xlabel("Author")
plt.ylabel("Count")
plt.title("Comparison of num_words and num_tokens by Author")
plt.legend(title="Metric")
plt.savefig(output_path_plots + f"words_tokens-{nb_id}.png")
plt.show()
```



```
[12]: df["token_word_ratio"] = df["num_tokens"] / df["num_words"]
      # Print token word ratio by author
      print(df.groupby("author")["token_word_ratio"].mean())
```

```
author
AT_Murray    0.43
Fagles       0.50
Fitzgerald   0.49
Green        0.47
Lattimore    0.44
Wilson       0.49
Name: token_word_ratio, dtype: float64
```

```
[13]: baseline_author = "AT_Murray"
      baseline_ratio = df[df["author"] == baseline_author]["token_word_ratio"].mean()
```

```
[14]: # Compute mean token-word ratio per author
      author_means = df.groupby("author")["token_word_ratio"].mean().reset_index()

      # Bar plot
      sns.barplot(data=author_means, x="author", y="token_word_ratio", hue="author",
                  palette=palette)

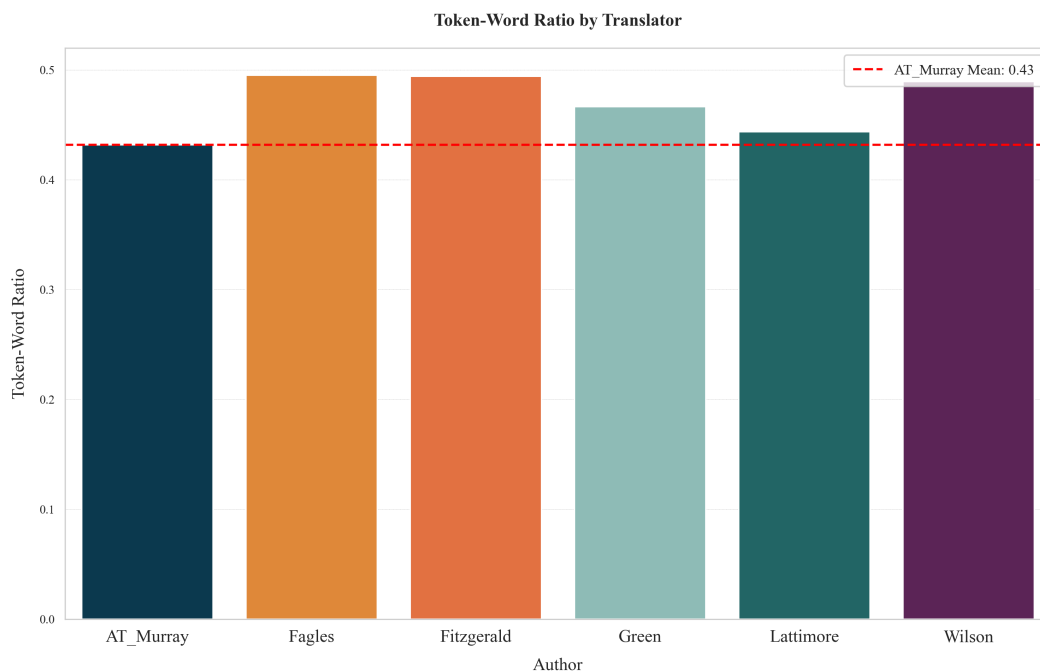
      # Baseline (horizontal line for reference author)
```

```

plt.axhline(baseline_ratio, color="red", linestyle="--",
            label=f"{baseline_author} Mean: {baseline_ratio:.2f}")

# Labels and title
plt.xlabel("Author")
plt.ylabel("Token-Word Ratio")
plt.title("Token-Word Ratio by Translator")
plt.xticks()
plt.legend()
plt.savefig(output_path_plots + f"token_word_ratio-{nb_id}.png") # Save plot
# Show plot
plt.show()

```



b) Token quality check

- DataFrame of firstlines (books, authors)
- Visual auditing of the Proems by author
- Purity index
 - by book
 - aggregated

```

[15]: df["first_line"] = df["text"].apply(lambda x: x[0] if isinstance(x, list) and
            len(x) > 0 else "")

```

```
first_line_df = df.groupby("author").first().reset_index()[["author",  
↪ "first_line"]]  
first_line_df
```

```
[15]:  author      first_line  
0    AT_Murray      Tell me, O Muse, of that many-sided hero who\n  
1      Fagles      Sing to me of the man, Muse, the man of twists and turns ...\n  
2 Fitzgerald      Sing in me, Muse, and through me tell the story \n  
3      Green  The man, Muse-tell me about that resourceful man, who wandered\n  
4 Lattimore      Tell me, Muse, of the man of many ways, who was driven\n  
5      Wilson      Tell me about a complicated man.\n
```

```
[16]: # Step 1: Convert the 'text' column from a list to a string and remove \n  
↪ characters  
df['text_string'] = df['text'].apply(lambda x: "".join(x).replace("\n", "") if  
↪ isinstance(x, list) else "")  
  
# Step 2: Split the string into sentences based on period (.)  
df['sentences'] = df['text_string'].apply(lambda x: x.split(".") if  
↪ isinstance(x, str) else [])  
  
# Step 3: Extract the first sentence from the 'sentences' list  
df['first_sentence'] = df['sentences'].apply(lambda x: x[0].strip() if len(x) >  
↪ 0 else "")  
  
# Step 4: Group by 'author' and extract the first sentence for each author  
first_sent_df = df.groupby("author").first().reset_index()[["author",  
↪ "first_sentence"]]  
  
# Check the result  
pd.set_option("display.max_colwidth", 400) # Prevent truncation of long values  
first_sent_df
```

```
[16]:  author      first_sentence  
0    AT_Murray  
Tell me, O Muse, of that many-sided hero whotraveled far and wide after he had  
sacked the famous town of Troy  
1      Fagles  
Sing to me of the man, Muse, the man of twists and turns ...driven time and again  
off course, once he had plunderedthe hallowed heights of Troy  
2 Fitzgerald  
Sing in me, Muse, and through me tell the story of that man skilled in all ways  
of contending, the wanderer, harried for years on end, after he plundered the  
stronghold on the proud height of Troy  
3      Green  The man, Muse-tell me about that resourceful man, who wanderedfar  
and wide, when he'd sacked Troy's sacred citadel:many men's townships he saw,  
and learned their ways of thinking,many the griefs he suffered at heart on the
```

```

open sea,battling for his own life and his comrades' homecoming
4   Lattimore
Tell me, Muse, of the man of many ways, who was drivenfar journeys, after he had
sacked Troy's sacred citadel
5       Wilson
Tell me about a complicated man

```

```
[17]: first_sent_df["first_sentence"][2]
```

```
[17]: 'Sing in me, Muse, and through me tell the story of that man skilled in all ways
of contending, the wanderer, harried for years on end, after he plundered the
stronghold on the proud height of Troy'
```

```
[18]: def extract_first_lines(df):
        """
        Extracts the first 9 lines of Book 1 for each author.

        Args:
            df (pd.DataFrame): The input DataFrame containing columns 'author',
            ↪ 'book_num', and 'text'.

        Returns:
            pd.DataFrame: A new DataFrame with authors and their first 9 lines from
            ↪ Book 1.
        """
        first_lines_data = []

        for author in df["author"].unique(): # Loop through each unique author
            book1_text = df.loc[(df["author"] == author) & (df["book_num"] == 1),
            ↪ "text"].values

            if len(book1_text) > 0: # Ensure there's data
                first_9_lines = "".join(book1_text[0][:9]) # Get first 9 lines
                first_lines_data.append({"author": author, "proem": first_9_lines})

        # Convert the list to a DataFrame
        df_proem = pd.DataFrame(first_lines_data)
        return df_proem

pd.set_option("display.max_colwidth", 800) # Prevent truncation of long values
df_proem = extract_first_lines(df)
df_proem
```

```
[18]: author      proem
0   AT_Murray Tell me, O Muse, of that many-sided hero who\ntraveled far and
wide after he had sacked the famous town of Troy . Many cities did he visit, and
many were\nthe people with whose customs and thinking \nhe was acquainted; many
```

things he suffered at sea while seeking to save his own\nlife and to achieve the safe homecoming\nof his companions; but do what he might\nhe could not save his men, for they perished through their own sheer\nrecklessness in eating the cattle of the Sun-god Helios; so the god prevented\nthem from ever reaching home. Tell me, as you have told those who came before\n

1 Fitzgerald

Sing in me, Muse, and through me tell the story \nof that man skilled in all ways of contending, \nthe wanderer, harried for years on end, \nafter he plundered the stronghold \non the proud height of Troy. \nHe saw the townlands \nand learned the minds of many distant men, \nand weathered many bitter nights and days \nin his deep heart at sea, while he fought only \n

2 Lattimore

Tell me, Muse, of

the man of many ways, who was driven\nfar journeys, after he had sacked Troy's sacred citadel.\nMany were they whose cities he saw, whose minds he learned of,\nmany the pains he suffered in his spirit on the wide sea,\nstruggling for his own life and the homecoming of his companions.\nEven so he could not save his companions, hard though\nhe strove to; they were destroyed by their own wild recklessness, fools,\nwho devoured the oxen of Helios, the Sun God, and he took away the day\nof their homecoming. From some point\n

3 Fagles

Sing to me of the man, Muse, the man of twists and turns ...\ndriven time and again off course, once he had plundered\nthe hallowed heights of Troy.\nMany cities of men he saw and learned their minds,\nmany pains he suffered, heartsick on the open sea,\nfighting to save his life and bring his comrades home.\nBut he could not save them from disaster, hard as he strove-\nthe recklessness of their own ways destroyed them all,\nthe blind fools, they devoured the cattle of the Sun\n

4 Wilson

Tell me about a complicated man.\nMuse, tell me how he wandered and was lost\nwhen he had wrecked the holy town of Troy,\nand where he went, and who he met, the pain\nhe suffered in the storms at sea, and how\nhe worked to save his life and bring his men\nback home. He failed to keep them safe; poor fools,\nthey ate the Sun God's cattle, and the god\nkept them from home. Now goddess, child of Zeus,\n

5 Green

The man, Muse-tell me about that

resourceful man, who wandered\nfar and wide, when he'd sacked Troy's sacred citadel:\nmany men's townships he saw, and learned their ways of thinking,\nmany the griefs he suffered at heart on the open sea,\nbattling for his own life and his comrades' homecoming. Yet\nno way could he save his comrades, much though he longed to-\nit was through their own blind recklessness that they perished,\nthe fools, for they slaughtered the cattle of Helios the sun god\nand ate them: for that he took from them their day of returning.\n

```
[19]: for author in df["author"].unique(): # Loop through each unique author
      book1_text = df[(df["author"] == author) & (df["book_num"] == 1)]["text"].
      ↪values
```



```
if len(book1_text) > 0: # Ensure there's data
    first_nine_lines = "".join(book1_text[0][:9]) # Get first 9 lines
    print(f"Author: {author}\n{first_nine_lines}\n{'-'*40}") # Print result
```

Author: AT_Murray

Tell me, O Muse, of that many-sided hero who
traveled far and wide after he had sacked the famous town of Troy . Many cities
did he visit, and many were
the people with whose customs and thinking
he was acquainted; many things he suffered at sea while seeking to save his own
life and to achieve the safe homecoming
of his companions; but do what he might
he could not save his men, for they perished through their own sheer
recklessness in eating the cattle of the Sun-god Helios; so the god prevented
them from ever reaching home. Tell me, as you have told those who came before

Author: Fitzgerald

Sing in me, Muse, and through me tell the story
of that man skilled in all ways of contending,
the wanderer, harried for years on end,
after he plundered the stronghold
on the proud height of Troy.
He saw the townlands
and learned the minds of many distant men,
and weathered many bitter nights and days
in his deep heart at sea, while he fought only

Author: Lattimore

Tell me, Muse, of the man of many ways, who was driven
far journeys, after he had sacked Troy's sacred citadel.
Many were they whose cities he saw, whose minds he learned of,
many the pains he suffered in his spirit on the wide sea,
struggling for his own life and the homecoming of his companions.
Even so he could not save his companions, hard though
he strove to; they were destroyed by their own wild recklessness, fools,
who devoured the oxen of Helios, the Sun God, and he took away the day
of their homecoming. From some point

Author: Fagles

Sing to me of the man, Muse, the man of twists and turns ...
driven time and again off course, once he had plundered
the hallowed heights of Troy.
Many cities of men he saw and learned their minds,
many pains he suffered, heartsick on the open sea,
fighting to save his life and bring his comrades home.

But he could not save them from disaster, hard as he strove-
the recklessness of their own ways destroyed them all,
the blind fools, they devoured the cattle of the Sun

Author: Wilson

Tell me about a complicated man.
Muse, tell me how he wandered and was lost
when he had wrecked the holy town of Troy,
and where he went, and who he met, the pain
he suered in the storms at sea, and how
he worked to save his life and bring his men
back home. He failed to keep them safe; poor fools,
they ate the Sun God's cattle, and the god
kept them from home. Now goddess, child of Zeus,

Author: Green

The man, Muse-tell me about that resourceful man, who wandered
far and wide, when he'd sacked Troy's sacred citadel:
many men's townships he saw, and learned their ways of thinking,
many the griefs he suffered at heart on the open sea,
battling for his own life and his comrades' homecoming. Yet
no way could he save his comrades, much though he longed to-
it was through their own blind recklessness that they perished,
the fools, for they slaughtered the cattle of Helios the sun god
and ate them: for that he took from them their day of returning.

[]:

1.4 Experiment 1: Type-Token Ratio (TTR)

1.5 Introduction

Type-Token Ratio (TTR) is a key measure of lexical diversity, calculated as the number of unique words (**types**) divided by the total word count (**tokens**). It provides insight into a translator's lexical choices when rendering the same source text.

- A **higher TTR** suggests a richer vocabulary, possibly reflecting an effort to capture nuances or stylistic complexity.
- A **lower TTR** may indicate a more repetitive or constrained word choice, potentially prioritizing accessibility or fidelity to the original.

1.5.1 TTR Formula

$$TTR = \left(\frac{\text{Unique Words}}{\text{Total Words}} \right) \times 100$$

```
[20]: import scipy.stats as stats

# Compute TTR
df["ttr"] = df["tokens"].apply(lambda x: (len(set(x)) / len(x) * 100) if x else 0)
```

```
[21]: # List of translators
translators = ["AT_Murray", "Fitzgerald", "Lattimore", "Fagles", "Wilson",
               "Green"]
ttr_by_translator = {}
for translator in translators:
    ttr_by_translator[translator] = df[df["author"] == translator]["ttr"].
    tolist()

# TTR DF by translator
if 'book_num' not in df.columns:
    df['book_num'] = [f"Book_num_{i+1}" for i in range(1, 25)] *
    len(translators)
ttr_df = df.pivot(index='book_num', columns='author', values='ttr')

# Reorder columns if needed
ttr_df = ttr_df[translators]
e.check_df(ttr_df)
```

No missing values

```
df columns: Index(['AT_Murray', 'Fitzgerald', 'Lattimore', 'Fagles', 'Wilson',
                  'Green'], dtype='object', name='author')
```

Shape: (24, 6)

```
[22]: from scipy import stats

for translator in translators:
    # Get the TTR data for this translator
    ttr_data = ttr_by_translator[translator]

    # Perform Shapiro-Wilk test
    stat, p_value = stats.shapiro(ttr_data)

    # Print results
    print(f"Shapiro-Wilk test for \n{translator}'s data: T-statistic={stat:.
    4f}, p-value={p_value:.4f}")
```

```

# Interpret results
if p_value < 0.05:
    print(f"{translator}'s TTR data is NOT normally distributed.")
else:
    print(f"{translator}'s TTR data is NORMALLY distributed.")

print() # Add empty line for readability

```

Shapiro-Wilk test for
 AT_Murray's data: T-statistic=0.9678, p-value=0.6131
 AT_Murray's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
 Fitzgerald's data: T-statistic=0.9609, p-value=0.4569
 Fitzgerald's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
 Lattimore's data: T-statistic=0.9699, p-value=0.6656
 Lattimore's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
 Fagles's data: T-statistic=0.9818, p-value=0.9273
 Fagles's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
 Wilson's data: T-statistic=0.9665, p-value=0.5814
 Wilson's TTR data is NORMALLY distributed.

Shapiro-Wilk test for
 Green's data: T-statistic=0.9691, p-value=0.6441
 Green's TTR data is NORMALLY distributed.

```

[23]: from itertools import combinations

# Use the ttr_by_translator dictionary
translator_names = translators # Use your existing list of translators
ttr_values = [ttr_by_translator[translator] for translator in translator_names]

# Perform one-way ANOVA
f_stat, p_value = stats.f_oneway(*ttr_values)
print(f"F-statistic: {f_stat:.4f}, P-value: {p_value:.4f}")

if p_value < 0.05:
    print("There are statistically significant differences in TTR among the_
    ↪ translators.")

```

```

else:
    print("There are no statistically significant differences in TTR among the_
    ↪translators.")

# Perform pairwise t-tests with Bonferroni correction
print("\nPairwise comparisons:")

# Number of comparisons for Bonferroni correction
num_comparisons = len(list(combinations(range(len(translator_names)), 2)))

for i, j in combinations(range(len(translator_names)), 2):
    t_stat, p_val = stats.ttest_ind(ttr_values[i], ttr_values[j])

    # Apply Bonferroni correction
    adj_p_val = min(p_val * num_comparisons, 1.0)

    # Calculate mean difference
    mean_diff = np.mean(ttr_values[i]) - np.mean(ttr_values[j])

    # Determine significance
    is_significant = "SIGNIFICANT" if adj_p_val < 0.05 else "NOT significant"

    print(f"{translator_names[i]} vs {translator_names[j]}: Diff = {mean_diff:.
    ↪4f}, p = {adj_p_val:.4f} - {is_significant}")

```

F-statistic: 9.5715, P-value: 0.0000

There are statistically significant differences in TTR among the translators.

Pairwise comparisons:

AT_Murray vs Fitzgerald: Diff = 0.0056, p = 1.0000 - NOT significant
 AT_Murray vs Lattimore: Diff = 0.0301, p = 0.0168 - SIGNIFICANT
 AT_Murray vs Fagles: Diff = 0.0281, p = 0.0340 - SIGNIFICANT
 AT_Murray vs Wilson: Diff = -0.0200, p = 0.8576 - NOT significant
 AT_Murray vs Green: Diff = 0.0261, p = 0.0695 - NOT significant
 Fitzgerald vs Lattimore: Diff = 0.0245, p = 0.1274 - NOT significant
 Fitzgerald vs Fagles: Diff = 0.0225, p = 0.2312 - NOT significant
 Fitzgerald vs Wilson: Diff = -0.0256, p = 0.2791 - NOT significant
 Fitzgerald vs Green: Diff = 0.0206, p = 0.4132 - NOT significant
 Lattimore vs Fagles: Diff = -0.0020, p = 1.0000 - NOT significant
 Lattimore vs Wilson: Diff = -0.0501, p = 0.0001 - SIGNIFICANT
 Lattimore vs Green: Diff = -0.0039, p = 1.0000 - NOT significant
 Fagles vs Wilson: Diff = -0.0481, p = 0.0001 - SIGNIFICANT
 Fagles vs Green: Diff = -0.0019, p = 1.0000 - NOT significant
 Wilson vs Green: Diff = 0.0462, p = 0.0003 - SIGNIFICANT

```

[24]: def cohens_d(group1, group2):
        n1, n2 = len(group1), len(group2)

```

```

var1, var2 = np.var(group1, ddof=1), np.var(group2, ddof=1)
pooled_std = np.sqrt(((n1 - 1) * var1 + (n2 - 1) * var2) / (n1 + n2 - 2))
mean_diff = np.mean(group1) - np.mean(group2)
return mean_diff / pooled_std

# Calculate Cohen's d for significant differences
significant_pairs = [
    ('Lattimore', 'Wilson'),
    ('Fagles', 'Wilson'),
    ('Wilson', 'Green'),
    ('Fitzgerald', 'Wilson'),
    ('Fitzgerald', 'Green'),
    ('Fitzgerald', 'Lattimore'),
    ('Fitzgerald', 'Fagles'),
    ('Lattimore', 'Fagles'),
    ('Lattimore', 'Green'),
    ('Fagles', 'Green')
]

for pair in significant_pairs:
    translator1, translator2 = pair
    group1 = ttr_by_translator[translator1]
    group2 = ttr_by_translator[translator2]

    d = cohens_d(group1, group2)
    t_stat, p_value = stats.ttest_ind(group1, group2)

    print(f"{translator1} vs {translator2}:")
    print(f"Cohen's d: {d:.4f}")
    print(f"T-statistic: {t_stat:.4f}")
    print(f"P-value: {p_value:.4e}")

    # Interpret effect size
    if abs(d) < 0.2:
        interpretation = "small"
    elif abs(d) < 0.5:
        interpretation = "medium"
    else:
        interpretation = "large"

    print(f"Effect size interpretation: {interpretation}")
    print()

```

Lattimore vs Wilson:
 Cohen's d: -1.4998
 T-statistic: -5.1955
 P-value: 4.5488e-06
 Effect size interpretation: large

Fagles vs Wilson:
Cohen's d: -1.4355
T-statistic: -4.9728
P-value: 9.6309e-06
Effect size interpretation: large

Wilson vs Green:
Cohen's d: 1.3652
T-statistic: 4.7292
P-value: 2.1676e-05
Effect size interpretation: large

Fitzgerald vs Wilson:
Cohen's d: -0.7043
T-statistic: -2.4399
P-value: 1.8604e-02
Effect size interpretation: large

Fitzgerald vs Green:
Cohen's d: 0.6570
T-statistic: 2.2760
P-value: 2.7546e-02
Effect size interpretation: large

Fitzgerald vs Lattimore:
Cohen's d: 0.7938
T-statistic: 2.7498
P-value: 8.4950e-03
Effect size interpretation: large

Fitzgerald vs Fagles:
Cohen's d: 0.7264
T-statistic: 2.5162
P-value: 1.5415e-02
Effect size interpretation: large

Lattimore vs Fagles:
Cohen's d: -0.0729
T-statistic: -0.2524
P-value: 8.0184e-01
Effect size interpretation: small

Lattimore vs Green:
Cohen's d: -0.1409
T-statistic: -0.4880
P-value: 6.2787e-01
Effect size interpretation: small

Fagles vs Green:
Cohen's d: -0.0687
T-statistic: -0.2380
P-value: 8.1292e-01
Effect size interpretation: small

```
[25]: def additional_metrics(group1, group2):  
        u3 = stats.norm.cdf(d).mean() * 100 # Percentage above average  
        overlap = 2 * stats.norm.cdf(-abs(d)/2) * 100 # Visual overlap  
        return u3, overlap  
additional_metrics(group1, group2)
```

```
[25]: (np.float64(47.26097167890923), np.float64(97.25935538648754))
```

```
[26]: def cohen_d(x, y):  
        nx, ny = len(x), len(y)  
        dof = nx + ny - 2  
        pooled_std = np.sqrt(((nx-1)*np.std(x, ddof=1)**2 + (ny-1)*np.std(y,   
        ↪ddof=1)**2) / dof)  
        return (np.mean(x) - np.mean(y)) / pooled_std  
  
        # Calculate effect sizes for significant differences  
        print("Effect sizes (Cohen's d):")  
        print(f"Lattimore vs Wilson: {cohen_d(ttr_by_translator['Lattimore'],   
        ↪ttr_by_translator['Wilson']):.4f}")  
        print(f"Fagles vs Wilson: {cohen_d(ttr_by_translator['Fagles'],   
        ↪ttr_by_translator['Wilson']):.4f}")  
        print(f"Wilson vs Green: {cohen_d(ttr_by_translator['Wilson'],   
        ↪ttr_by_translator['Green']):.4f}")
```

Effect sizes (Cohen's d):
Lattimore vs Wilson: -1.4998
Fagles vs Wilson: -1.4355
Wilson vs Green: 1.3652

```
[27]: import pandas as pd  
import statsmodels.formula.api as smf  
import statsmodels.api as sm  
  
        # Assuming your DataFrame is named 'df' and has columns 'author', 'book_num',   
        ↪and 'ttr'  
        # If not, adjust the column names accordingly  
  
        # Fit the mixed-effects model  
        # 'ttr' is the dependent variable  
        # 'author' is the fixed effect (the translators you want to compare)
```



```

# 'book_num' is the random effect (to account for book-to-book variation)

# The formula specifies the model
# C(author) tells the model that 'author' is a categorical variable
# The random intercept is specified as '1 | book_num'
model = smf.mixedlm("ttr ~ C(author)", data=df, groups=df["book_num"])

# Fit the model
results = model.fit()

# Print the results summary
print(results.summary())

# If you want to perform post-hoc tests, you can use the 'pairwise_tukeyhsd'
  ↳function
# from the statsmodels.stats.multicomp module. However, this function is not
# directly compatible with the mixed-effects model results. You would need to
# extract the estimated marginal means (least squares means) and their standard
  ↳errors
# from the mixed-effects model results and then use those values in the
  ↳'pairwise_tukeyhsd'
# function.

# The results summary will give you the estimated coefficients for each
  ↳translator,
# as well as standard errors, t-values, and p-values. You can use these values
# to compare the translators and determine whether there are significant
  ↳differences
# in TTR between them.

```

Mixed Linear Model Regression Results

=====						
Model:	MixedLM	Dependent Variable:		ttr		
No. Observations:	144	Method:		REML		
No. Groups:	24	Scale:		0.0000		
Min. group size:	6	Log-Likelihood:		450.2371		
Max. group size:	6	Converged:		Yes		
Mean group size:	6.0					

	Coef.	Std.Err.	z	P> z	[0.025	0.975]

Intercept	0.160	0.006	24.800	0.000	0.147	0.173
C(author) [T.Fagles]	-0.028	0.002	-17.388	0.000	-0.031	-0.025
C(author) [T.Fitzgerald]	-0.006	0.002	-3.446	0.001	-0.009	-0.002
C(author) [T.Green]	-0.026	0.002	-16.196	0.000	-0.029	-0.023
C(author) [T.Lattimore]	-0.030	0.002	-18.630	0.000	-0.033	-0.027
C(author) [T.Wilson]	0.020	0.002	12.404	0.000	0.017	0.023

Group	Var	0.001	0.056
=====			

```
/Users/debr/anaconda3/envs/odyssey-nlp/lib/python3.11/site-
packages/statsmodels/regression/mixed_linear_model.py:2237: ConvergenceWarning:
The MLE may be on the boundary of the parameter space.
  warnings.warn(msg, ConvergenceWarning)
```

```
[28]: e.check_df(df)
```

No missing values

```
df columns: Index(['author', 'book_num', 'text', 'tokens', 'num_words',
'num_tokens', 'diff', 'token_word_ratio', 'first_line', 'text_string',
'sentences', 'first_sentence', 'ttr'], dtype='object')
```

Shape: (144, 13)

```
[29]: df.drop(df[['first_sentence', 'sentences']], axis=1, inplace=True)
```

```
[30]: import pandas as pd
import numpy as np

# Assuming you have a DataFrame 'df' with columns 'author', 'book_num', 'text',
↳ and 'tokens'

# -----
# Standardized TTR (STTR)
# -----

def calculate_sttr(tokens, segment_size=100):
    """
    Calculates Standardized Type-Token Ratio (STTR).

    Args:
        tokens (list): List of tokens (words).
        segment_size (int): Size of each segment to calculate TTR on.

    Returns:
        float: STTR value.
    """
    if len(tokens) < segment_size:
        return len(set(tokens)) / len(tokens) * 100 # Fallback to regular TTR
↳ if too short

    num_segments = len(tokens) // segment_size
    ttr_values = []
```

```

    for i in range(num_segments):
        segment = tokens[i * segment_size: (i + 1) * segment_size]
        ttr = len(set(segment)) / len(segment) * 100
        ttr_values.append(ttr)

    return np.mean(ttr_values)

# Apply STTR calculation to your DataFrame
df['sttr'] = df['tokens'].apply(calculate_sttr)

# -----
# Moving-Average TTR
# -----

def calculate_moving_average_ttr(tokens, window_size=100):
    """
    Calculates Moving-Average Type-Token Ratio.

    Args:
        tokens (list): List of tokens (words).
        window_size (int): Size of the moving window.

    Returns:
        list: List of moving-average TTR values.
    """
    if len(tokens) < window_size:
        return [len(set(tokens)) / len(tokens) * 100] # Fallback to regular
↪TTR if too short

    moving_average_ttr = []
    for i in range(window_size, len(tokens) + 1):
        window = tokens[i - window_size:i]
        ttr = len(set(window)) / len(window) * 100
        moving_average_ttr.append(ttr)
    return moving_average_ttr

# Apply Moving-Average TTR calculation to your DataFrame
# This creates a new column with lists of TTR values for each book
df['moving_average_ttr'] = df['tokens'].apply(calculate_moving_average_ttr)

# -----
# Example Usage and Analysis
# -----

# Now you can analyze the STTR and moving_average_ttr

```

```

# For example, calculate the average STTR for each translator:
average_sttr_by_translator = df.groupby('author')['sttr'].mean()
print("Average STTR by translator:\n", average_sttr_by_translator)

# For moving average TTR, you might want to plot the TTR values over the course
↳ of each book

# Example: Print the first few moving average TTR values for the first book of
↳ AT_Murray
first_book_at_murray = df[(df['author'] == 'AT_Murray') & (df['book_num'] ==
↳ 1)]['moving_average_ttr'].iloc[0] #Adapt book_num
print("\nMoving Average TTR for first book of AT_Murray (first few values):\n",
↳ first_book_at_murray[:5])

```

Average STTR by translator:

author	
AT_Murray	21.50
Fagles	21.62
Fitzgerald	21.60
Green	21.65
Lattimore	21.60
Wilson	21.78

Name: sttr, dtype: float64

Moving Average TTR for first book of AT_Murray (first few values):
[23.0, 22.0, 22.0, 22.0, 22.0]

```

[31]: import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Explode the 'moving_average_ttr' column if it contains lists
if isinstance(df["moving_average_ttr"].iloc[0], list): # Check if the column
↳ contains lists
    df = df.explode("moving_average_ttr")

# Convert to float (just in case)
df["moving_average_ttr"] = df["moving_average_ttr"].astype(float)

# Sort values by author and book number to ensure correct order
df = df.sort_values(by=["author", "book_num"])

```

```

[32]: # Set figure size
plt.figure(figsize=(12, 6))

# Line plot: Moving Average TTR by author

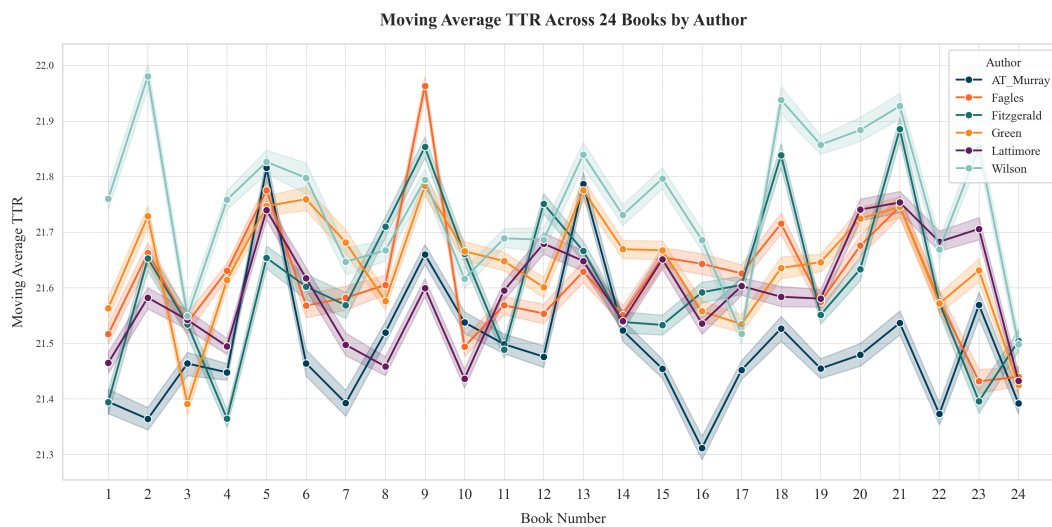
```

```

sns.lineplot(data=df, x="book_num", y="moving_average_ttr", hue="author",
             ↪marker="o")

# Customize labels and title
plt.xlabel("Book Number", fontsize=12)
plt.ylabel("Moving Average TTR", fontsize=12)
plt.title("Moving Average TTR Across 24 Books by Author", fontsize=14)
plt.xticks(range(1, 25)) # Ensure x-axis labels from 1 to 24
plt.legend(title="Author") # Legend showing each author's line
plt.grid(True, linestyle="--", alpha=0.5) # Add a grid for clarity
plt.tight_layout() # Adjust layout for better appearance
plt.savefig(output_path_plots + f"moving_average_ttr-{nb_id}.png")
# Show plot
plt.show()

```



[]:

[]: